Walkthrough lead form – how it works end to end

Date: 6 Nov 2026

Project: Colive website rebuild (Next.js, `website-react`), branch `QADev-Hari`

Section: home page, "Get a personalised Colive walkthrough"

In short

A visitor enters a name and a WhatsApp number on the home page. The form is checked in the browser and then submitted as a normal page submit (not AJAX) to `/api/Offers/LeadCapture` in this app. That route checks the fields again and forwards them to the legacy website's endpoint of the same name (`https://qa.colive.com/api/Offers/LeadCapture` on QA). The legacy site saves the lead in the CRM. The visitor then lands back on the home page at `/?act=thankyou?dt=…&name=…`, where the form is replaced by "Thanks {name} — we'll WhatsApp you soon."

The legacy endpoint only saves a lead when the request has a lowercase `origin` header equal to the site's own URL. See section 5. This was the cause of the "Something went wrong" errors during development.

1. Files

File	Role
`src/components/home/WalkthroughSection.tsx`	The section: heading, subtitle, disclaimer. Chooses form or thank-you message inside `<Suspense>`.
`src/components/home/WalkthroughForm.tsx`	`WalkthroughForm`: inputs, validation, hidden fields. `WalkthroughResult`: reads the URL and shows the form, the error or the thank-you message.
`src/app/api/Offers/LeadCapture/route.ts`	Server route that receives the form post, checks it, forwards it to the legacy site and redirects the visitor.
`src/lib/leadCapture.ts`	Shared phone rule (`LEAD_PHONE_PATTERN`) and tracking field names (`LEAD_TRACKING_KEYS`).
`src/lib/server/apiConfig.ts`	QA/LIVE switch. The `site` service gives the legacy website URL.
`.env.example`	Optional overrides: `COLIVE_QA_SITE_URL`, `COLIVE_LIVE_SITE_URL`.
`src/app/page.tsx`	Places `<WalkthroughSection />` on the home page.

Legacy (.NET) code it depends on, in the qa.colive.com source: `OffersController.LeadCapture()` and `ApiControllerBase.PostAPICall()`.

2. End-to-end flow

```
[Browser – home page]      1-3  fill in, validate, copy gclid/utm_* into hidden fields
        |
        |  4  native POST /api/Offers/LeadCapture  (form-urlencoded)
        v
[Next.js route]            5  re-check name + phone
        |
        |  6  POST {site}/api/Offers/LeadCapture  + header "origin: {site origin}"
        v
[Legacy site]              OffersController.LeadCapture() -> PostAPICall (checks origin)
        |
        |  7  POST CRMapi/PropertyCRM/LandingPageLeadInsertWithScheduleVisit
        v
[CRM]                      saves the lead
        |
        |  8  legacy site answers 302 -> {site}/?act=thankyou?dt=yyyyMMddHHmmss&name=...
        v
[Next.js route]            9  answers 303 -> /?act=thankyou?dt=...&name=...#walkthrough (our domain)
        |
        v
[Browser – home page]      10  shows "Thanks {name} — we'll WhatsApp you soon."
```

Step by step:

1. **Fill in.** The visitor types a full name and a WhatsApp number.
2. **Browser validation.** `onSubmit` in `WalkthroughForm` checks both fields (rules in section 4). If either is invalid, it cancels the submit and shows the message under that field. Each message clears as soon as its field becomes valid.
3. **Tracking fields.** If valid, the form copies `gclid`, `utm_source`, `utm_medium`, `utm_campaign`, `utm_content`, `utm_term` and `utm_keyword` from the page's address (for example `/?utm_source=google&gclid=…`) into hidden inputs. Missing values are sent empty.
4. **Submit.** The browser posts the form normally (`method="post"`, `action="/api/Offers/LeadCapture"`), as the legacy site does. No JavaScript fetch is involved, so the browser follows the redirects itself.
5. **Server check.** The route reads the form data, trims the name and number, and runs the same checks again. Invalid data goes straight to the error state, so a request that skips the browser checks can't reach the CRM.
6. **Forward.** The route posts every field, form-urlencoded, to `{site}/api/Offers/LeadCapture`, where `{site}` comes from `apiConfig.ts`. It adds `origin: {site origin}` and asks `fetch` not to follow redirects (`redirect: "manual"`), so it can read the legacy response.
7. **Save.** The legacy controller builds a `CustomerDto` (name, mobile, lead source, UTM values, landing page ID, platform) and calls the CRM endpoint `CRMapi/PropertyCRM/LandingPageLeadInsertWithScheduleVisit`.
8. **Legacy redirect.** It answers `302 Found` with `Location: https://qa.colive.com/?act=thankyou?dt=20261106122907&name=Hariprasad`.
9. **Our redirect.** The route keeps only the path and query from that `Location`, adds `#walkthrough`, and redirects the visitor with `303 See Other` on our own domain. Dropping the host means the visitor never leaves our site.
10. **Thank-you state.** The home page loads. `WalkthroughResult` reads the URL with `useSearchParams()`. Because `act` starts with `thankyou`, it shows the thank-you message with the name and scrolls it into view.

3. Fields posted

Field	Value	Source
`first_name`	Full name as typed	Visible input
`customer_mobile`	WhatsApp number as typed	Visible input
`gclid`, `utm_source`, `utm_medium`, `utm_campaign`, `utm_content`, `utm_term`, `utm_keyword`	Same-named values from the page URL, or empty	Hidden inputs, filled on submit
`leadSource`	`Colive Home Page`	Hidden, fixed
`LandingPageId`	`1`	Hidden, fixed
`Mobile_Number_Validated`	`false`	Hidden, fixed
`redirectURL`	`/?act=thankyou`	Hidden, fixed. The legacy site appends `?dt=…&name=…` to it.

The field names and fixed values match the legacy home page form exactly. Don't rename them; the legacy controller reads them by name.

4. Validation rules

The same rules and messages as the legacy site's script:

Field	Rule	Message
Full name	Required after trimming spaces	"Please enter your name"
WhatsApp number	Required	"Please enter your WhatsApp number"
WhatsApp number	Must match `^\+?[\d\s-]{7,15}$`: 7–15 characters of digits, spaces or dashes, with an optional leading `+`	"Enter a valid WhatsApp number"

- The rule lives in `LEAD_PHONE_PATTERN` (`src/lib/leadCapture.ts`). The browser and the server route both use it, so they can't drift apart.
- The legacy HTML also carried `pattern="[6789][0-9]{10}"` on the input. The form has `novalidate`, so that attribute never applied, and it isn't used here either.

5. Legacy backend behaviour you need to know

The `origin` check (important)

`ApiControllerBase.PostAPICall()` only calls the CRM when this is true:

```
request.Headers.ToString().Contains("origin: " + _origin)   // _origin = Global.websiteURL without the trailing "/"
```

- The match is **case-sensitive**, including the header name. It must be exactly `origin: https://qa.colive.com` (QA) or `origin: https://www.colive.com` (LIVE).
- Browsers pass because Chrome uses HTTP/2, which always sends header names in lowercase.
- If the header is missing or capitalised (`Origin:`), `PostAPICall` skips the CRM and returns `null`. The controller then crashes casting that result, and the `catch` returns `200 OK` with `{"Status":"failed","Message":"Something went wrong!","Data":null}`. No lead is saved.

- Our route therefore sends `origin: site.origin` with the key written in **lowercase** in the code. Node's built-in `fetch` (Node 24) sends header names exactly as written, so `Origin:` would fail.

Any other legacy `/api/...` endpoint that goes through `PostAPICall` has the same requirement.

What `LeadCapture()` does

- Sets cookies for `utm_source`, `utm_campaign`, `utm_medium`, `utm_keyword` and `gclid`, scoped to `/api/Offers/LeadCapture`. They have no effect on our site.
- Builds the lead name from `customer_name`, or from `first_name` + `' '` + `last_name`. We send only `first_name`, so the CRM name has a trailing space; this is harmless.
- Calls `LandingPageLeadInsertWithScheduleVisit` (OTP verification is skipped because `IsOtpVerificationRequired` isn't sent).
- Always redirects to `{websiteURL}/{redirectURL}?dt={yyyyMMddHHmmss}&name={name without spaces}`. This happens even when the CRM insert itself returns a failure. Only an exception (such as the origin problem) produces the JSON error.

Quirks in the thank-you URL

- `redirectURL` already contains `?act=thankyou`, so the final URL has two `?`: `/?act=thankyou?dt=…&name=…`. Read normally, `act` is `thankyou?dt=20261106122907`. That's why the page checks `act.startsWith("thankyou")`.
- The legacy site removes spaces from the name: "Hari Prasad" is shown as "HariPrasad". Our page shows the name exactly as it appears in the URL, matching the live site.

6. Result states on the page

URL	What the section shows
`/` (no `act`)	The form
`/?act=thankyou?dt=…&name=Hari#walkthrough`	"Thanks Hari — we'll WhatsApp you soon." The form is hidden and the message is scrolled into view.
`/?act=leaderror#walkthrough`	"Something went wrong. Please try again." above the form. Added in this rebuild; the legacy site has no error handling.

The heading, subtitle and disclaimer stay visible in every state.

When the route returns `leaderror`:

- the name is empty, or the number fails the rule;
- the legacy site answers anything other than a redirect or a JSON `"Status": "success"`;
- the legacy site can't be reached (network error).

On the last two, the route writes a line in the server terminal:

```
LeadCapture failed: <status> <first 300 characters of the response>
```

Fallback: if the legacy site ever answers with JSON `"Status": "success"` instead of redirecting, the route builds the same thank-you URL itself.

Static page note: the home page is pre-built as static HTML. The thank-you message depends on the URL, so it is rendered in the browser, inside `<Suspense>`, with the plain form as the fallback. Fetching `/?act=thankyou…` with curl shows the form, not the message; that's expected.

7. QA and LIVE

The target follows the existing switch in `src/lib/server/apiConfig.ts`:

`COLIVE_ENV`	Legacy site used (`site` service)	Origin header sent
`QA` (this branch)	`https://qa.colive.com/`	`https://qa.colive.com`
`LIVE`	`https://www.colive.com/`	`https://www.colive.com`

Both URLs can be overridden with `COLIVE_QA_SITE_URL` and `COLIVE_LIVE_SITE_URL` in `.env.local`. The origin header is always derived from the URL in use, so it stays correct automatically.

On LIVE, every submission creates a real lead in the production CRM. Test with your own number and tell the sales team.

8. How to test

In the browser (localhost):

1. Run `npm run dev` and open `http://localhost:3000/`.
2. Submit with empty fields: both messages appear. Type a valid value: that message clears.
3. Enter `12ab` as the number: "Enter a valid WhatsApp number".
4. Submit valid details: the URL changes to `/?act=thankyou?dt=…&name=…` and the thank-you message shows.
5. Optional: open `/?utm_source=test&gclid=abc` first, submit, and check in DevTools (step below) that the hidden fields carried those values.

Inspecting the request in DevTools:

- Network tab → tick **Keep log** → choose **All** or **Doc** (not Fetch/XHR, because this is a normal page submit) → filter `LeadCapture`.
- **Payload** shows the posted fields.
- On localhost, **Headers** should show `303` with a `Location` of `/?act=thankyou…`.

From the command line (creates a real lead on QA; use clearly marked test data):

```
# Through our route (dev server on port 3000)
curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" -X POST http://localhost:3000/api/Offers/LeadCapture \
  --data-urlencode "first_name=Test (ignore)" --data-urlencode "customer_mobile=9999999999" \
  --data-urlencode "leadSource=Colive Home Page" --data "LandingPageId=1&Mobile_Number_Validated=false" \
  --data-urlencode "redirectURL=/?act=thankyou"
# Expected: 303 http://localhost:3000/?act=thankyou?dt=…&name=Test(ignore)#walkthrough

# Directly against the legacy site (note the lowercase origin header and HTTP/1.1)
curl -s -i --http1.1 -X POST https://qa.colive.com/api/Offers/LeadCapture -H "origin: https://qa.colive.com" \
  --data-urlencode "first_name=Test (ignore)" --data-urlencode "customer_mobile=9999999999" \
  --data-urlencode "leadSource=Colive Home Page" --data "LandingPageId=1&Mobile_Number_Validated=false" \
  --data-urlencode "redirectURL=/?act=thankyou"
# Expected: HTTP/1.1 302 Found, Location: https://qa.colive.com/?act=thankyou?dt=…
```

Test leads created during development, which can be deleted from QA: name "Claude Test (ignore)", number `9999999999`, two leads on 6 Nov 2026 around 12:28 IST.

9. Troubleshooting

Symptom	Likely cause	What to check
`?act=leaderror` and the log shows `LeadCapture failed: 200 {"Status":"failed","Message":"Something went wrong!"…}`	The legacy controller threw an exception, usually the `origin` check (section 5)	That the route still sends a lowercase `origin` key equal to the site origin. If it does, ask the backend team for the `LeadCapture()` entry in the legacy error log (`Global.utilityLog`) at that time.
`?act=leaderror` and the log shows a network error	The legacy site can't be reached	`getBaseUrl("site")` and the env overrides; whether qa.colive.com is up
`?act=leaderror` with no log line	The server-side check rejected the name or number	The posted values (DevTools → Payload)
Thank-you URL appears but no message	`act` value changed format	`WalkthroughResult` expects `act` to start with `thankyou`
The visitor ends up on qa.colive.com	The redirect handling was changed	The route must keep only `pathname + search` from `Location`
Lead saved but UTM values missing	The page URL had no UTM parameters, or the names differ	`LEAD_TRACKING_KEYS` and the page URL

10. Changing things later

- **Adding a field:** add the input to `WalkthroughForm` with the exact name the legacy controller reads (see `OffersController.LeadCapture()`, for example `customer_email` or `city`). The route forwards every field automatically.
- **Changing the phone rule:** edit `LEAD_PHONE_PATTERN` only; the browser and the server both use it.
- **Calling the CRM directly later:** if the legacy site is retired, replace the forward in `route.ts` with a call to `CRMapi/PropertyCRM/LandingPageLeadInsertWithScheduleVisit` (register it in `coliveEndpoints.ts`). Then build the `CustomerDto` fields the legacy controller currently fills, such as `TypeId=1`, `PageType=1` and the platform ID.